

Project 1

DATA 624

Kevin Havis

October 25, 2025

Project 1

Part A - ATM Forecast

Our goal is to model and forecast the cash balances of four different ATMs over time. The data contains daily cash balances for each ATM, with some missing values that need to be addressed.

Executive Summary

Simpler models appear to work the best for all of these ATMs, likely due to the consistent seasonal patterns in the data. Given the business context, I recommend ETS models for ATM 1 and ATM 2 due to their simplicity, while ATM 4 is best served by a seasonal naive model with drift. ATM 3 does not have sufficient data to train a robust model, so we recommend sourcing data from a similar machine or using a naive random walk until more data is available.

Do note that several missing data observations were found in the data; these were imputed to allow for proper model training, but should be reviewed by domain experts.

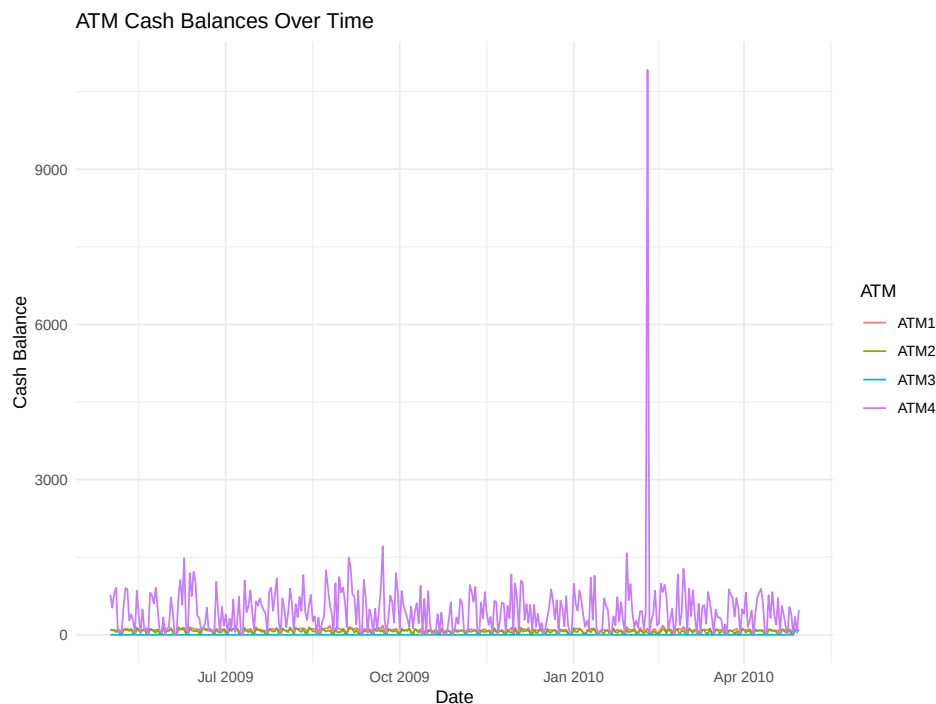
Exploratory Data Analysis

Examining the data, I see several entries with missing values at the end of the time series. Assuming these are the intended forecast periods, I will remove them from the raw data.

Plotting the remaining series, we have four ATMs. Of particular note is “ATM 4”, which seems to generally have much higher levels than the others, including a massive outlier.

```
# Read in ATM data
atm_data <- readxl::read_excel("ATM624Data.xlsx") |>
  mutate(DATE = as.Date(DATE, origin = "1899-12-30")) |>
  as_tsibble(index=DATE, key=ATM) |>
  drop_na()

autoplot(atm_data) +
  labs(title="ATM Cash Balances Over Time",
       y="Cash Balance",
       x="Date")
```



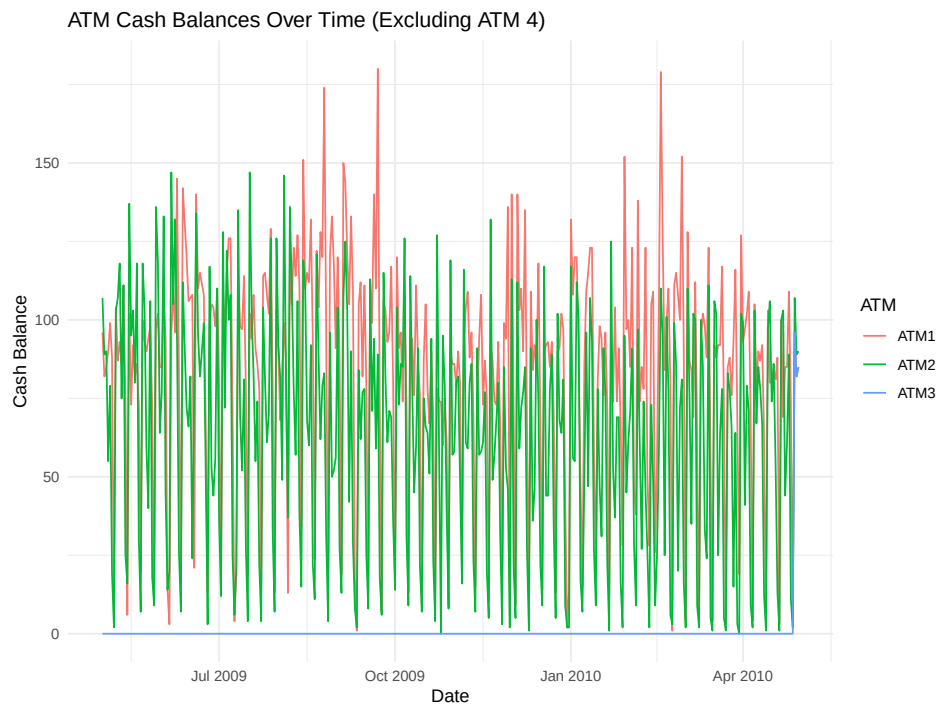
We temporarily remove ATM 4 in order to more clearly see the series of the remaining three.

ATM 3 also appears to be unusual, with zero balances through most the time series, then a sudden spike near the end.

ATM 1 and 2 demonstrate large variances of cash balances through the series, presumably showing the ATM's cash reserves gradually being drawn down, then suddenly replenished.

```
autoplot(atm_data |> filter(ATM != "ATM4")) +
  labs(title="ATM Cash Balances Over Time (Excluding ATM 4)",
```

```
y="Cash Balance",  
x="Date")
```



Given the differences, and assuming there's real business reasons behind them, we will analyze and model each ATM separately.

ATM 1

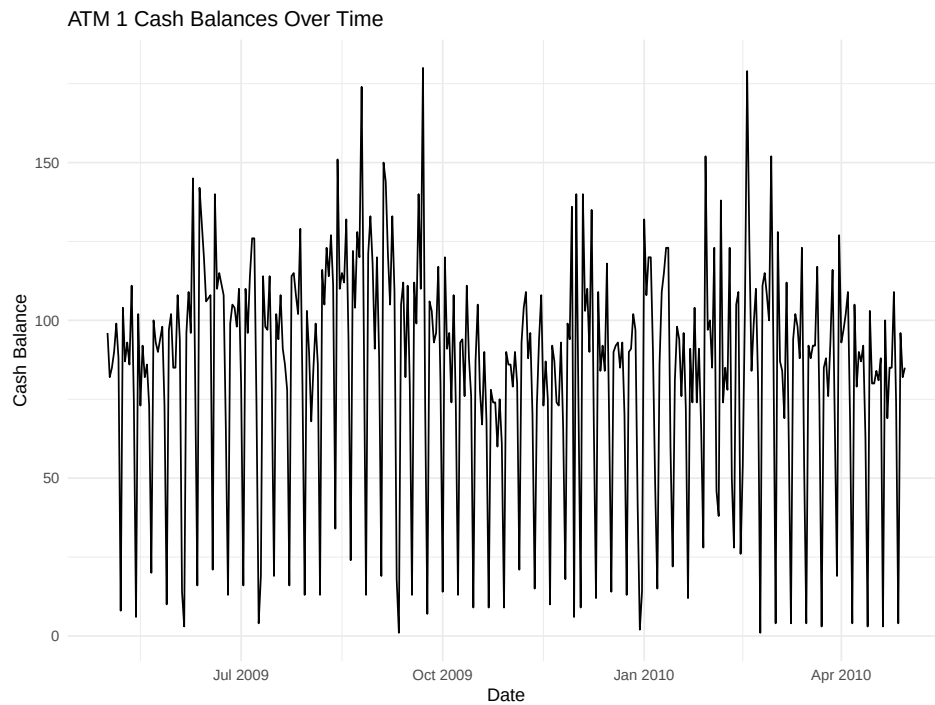
We will walk through the details of our model selection process for ATM 1, then summarize the results for the other ATMs, calling out any specific nuances as we go.

Reviewing the time series for ATM 1, we note the following characteristics:

- Variance is stable over time
- High degree of seasonality
- Minimal trend
- There are gaps in the time series we will need to address
 - Since the data is very seasonal, we'll simply use the seasonal lag to impute missing values
 - The seasonality is weekly

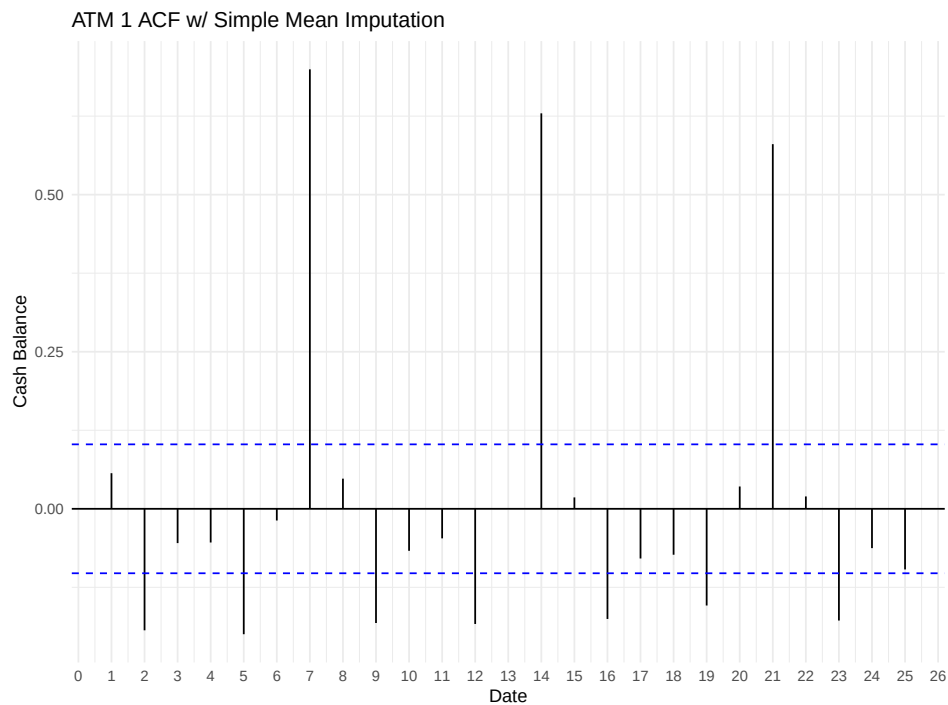
We will start by fixing the missing values. As mentioned, seasonal imputation seems reasonable given the consistency of the variance.

```
autoplot(atm_data |> filter(ATM == "ATM1")) +  
  labs(title="ATM 1 Cash Balances Over Time",  
        y="Cash Balance",  
        x="Date")
```



```
# Get ATM 1 data  
atm1 <- atm_data |>  
  filter(ATM == "ATM1")  
  
# Impute simple mean so we can check ACF  
atm1_fillmean <- atm1 |>  
  fill_gaps(Cash = mean(Cash, na.rm = TRUE))  
  
# Check what seasonality to impute on  
atm1_fillmean |>  
  ACF(Cash) |>  
  autoplot() +  
  scale_x_continuous(breaks = seq(0, 50, by = 1)) +
```

```
labs(
  title = "ATM 1 ACF w/ Simple Mean Imputation",
  y = "Cash Balance",
  x = "Date"
)
```



```
# Fill gaps using seasonal naive lag = 7
# We do this manually because snaive can't fit with gaps
atm1_filled <- atm1 %>%
  fill_gaps() %>%
  group_by_key() %>%
  mutate(Cash = if_else(is.na(Cash), lag(Cash, 7), Cash)) %>%
  ungroup()

# Confirm gaps are fixed
scan_gaps(atm1_filled) # should be empty
```

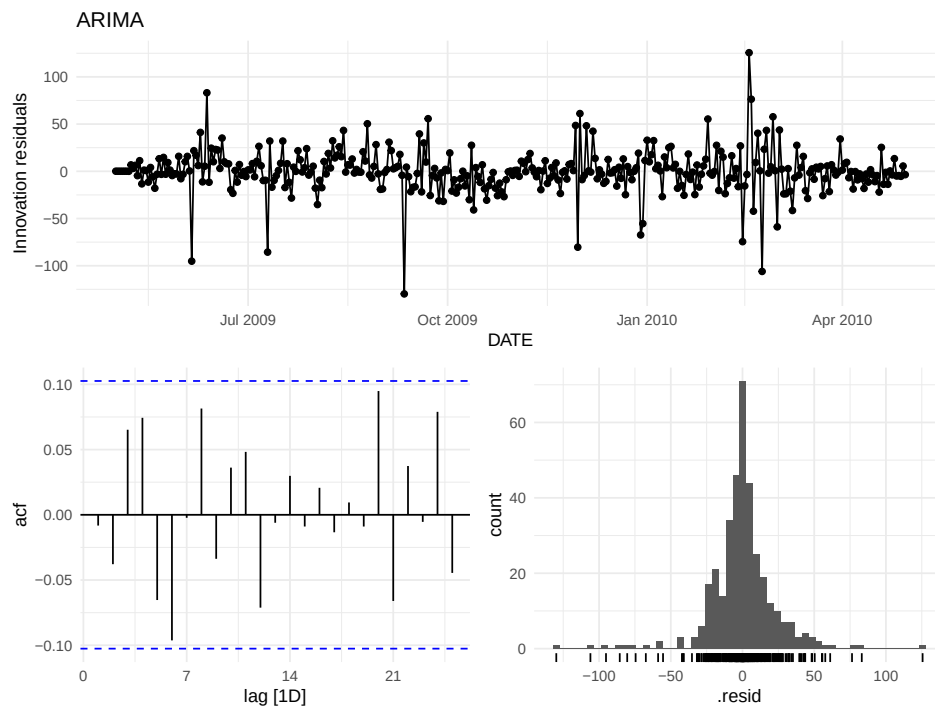
```
# A tsibble: 0 x 2 [?]
# Key:      ATM [0]
# i 2 variables: ATM <chr>, DATE <date>
```

We will select several models that deal with seasonality well and train them. Again, since the series is very consistent, we will compare naive models against something like ARIMA or ETS.

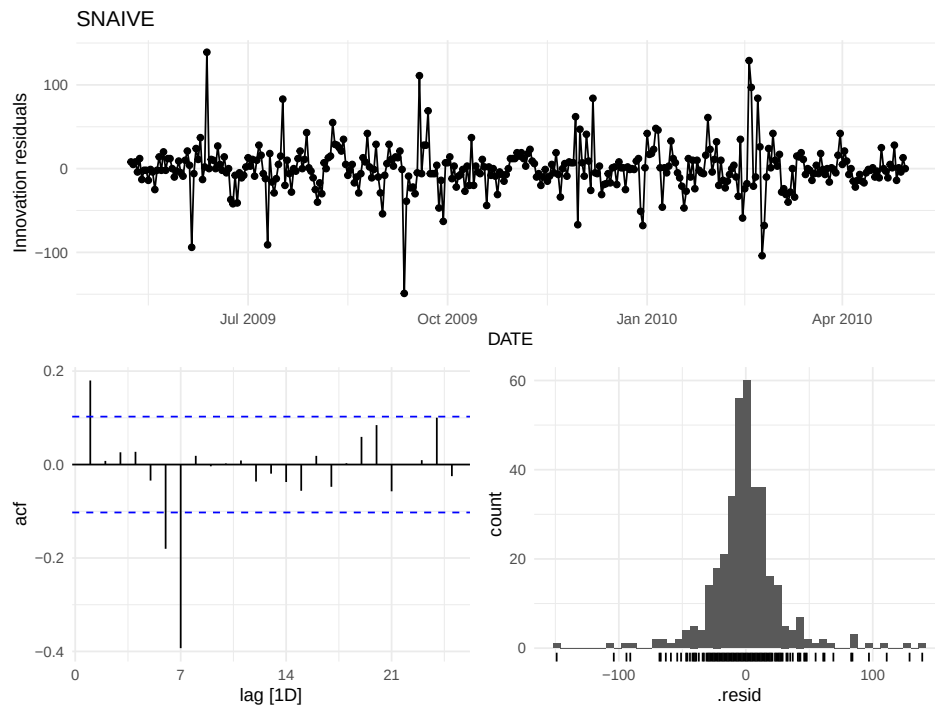
Below are the results of the point forecast for each model. ARIMA outperforms naive models but we will evaluate on a train/test split next.

```
atm1_fit <- atm1_filled |>
  #mutate(Cash = difference(Cash, lag=7)) |>
  model(arima = ARIMA(Cash ~ PDQ(period = 7)),
        snaive = SNAIVE(Cash ~ lag("week")),
        snaive_drift = SNAIVE(Cash ~ drift()),
        ets = ETS(Cash))

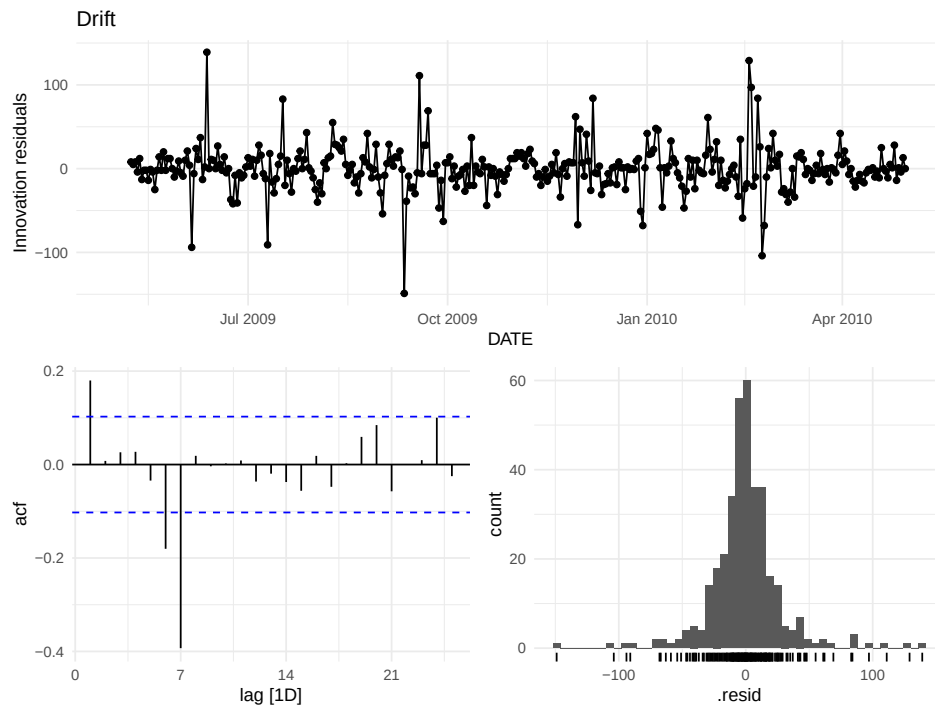
atm1_fit |>
  select(arima) |>
  gg_tsresiduals() + labs(title="ARIMA")
```



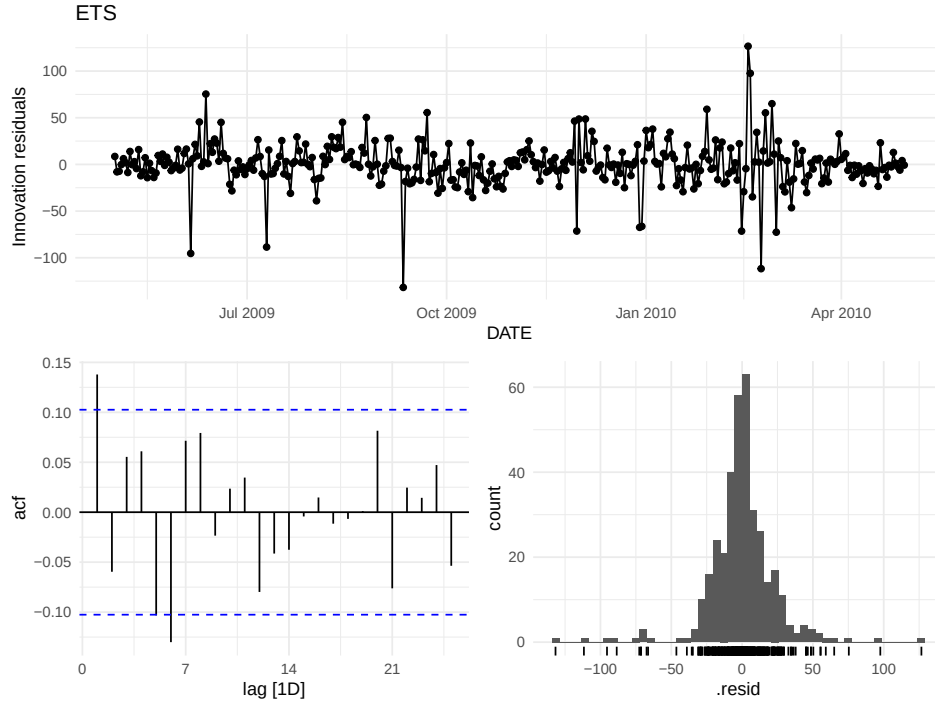
```
atm1_fit |>
  select(snaive) |>
  gg_tsresiduals() + labs(title="SNAIVE")
```



```
atm1_fit |>
  select(snaive_drift) |>
  gg_tsresiduals() + labs(title="Drift")
```



```
atm1_fit |>
  select(ets) |>
  gg_tsresiduals() + labs(title="ETS")
```

```
atm1_fit |>
  accuracy() |>
  select(-all_of(exclude_metrics)) |>
  kable(caption="ATM 1 Point Estimate Accuracy")
```

Table 1: ATM 1 Point Estimate Accuracy

ATM	.model	.type	RMSE	MASE	ACF1
ATM1	arima	Training	23.26277	0.8302375	-0.0082438
ATM1	snaive	Training	27.66050	1.0000000	0.1799236
ATM1	snaive_drift	Training	27.66048	1.0000231	0.1799236
ATM1	ets	Training	23.76149	0.8579801	0.1378691

Training on 90% of data, we've forecasted out 14 days to compare model performance.

ETS has the best MASE on the test data, and given the simplicity of the model compared to ARIMA, we would prefer it for this series. It also has the advantage of being more explainable to business stakeholders.

Recommendation for ATM 1: Deploy an ETS model and monitor for drift over time.

```

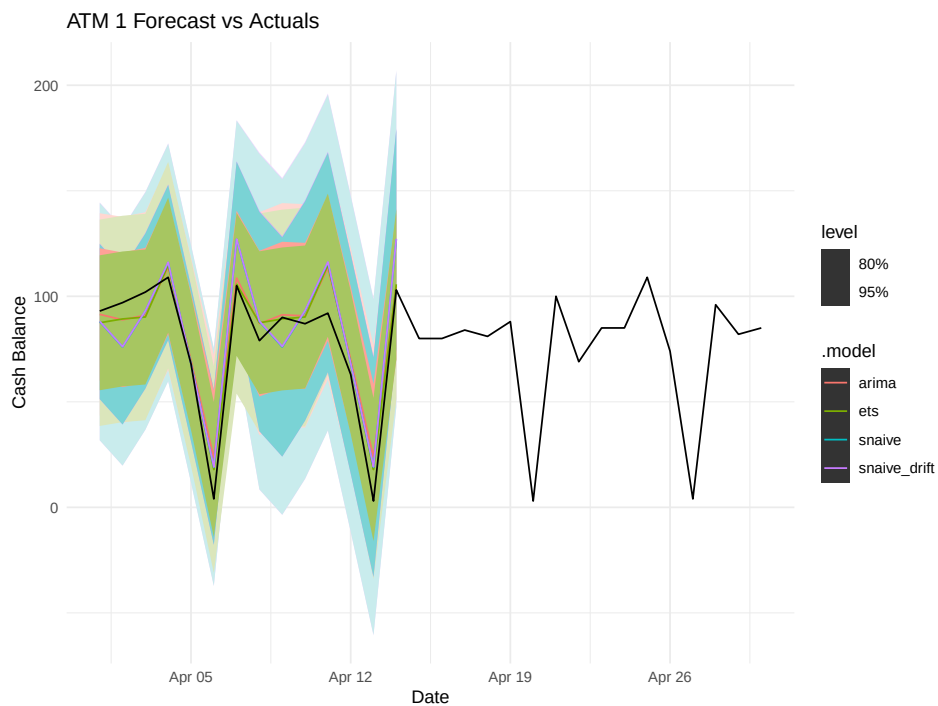
# Split the data into train and test
atm1_train <- atm1_filled |> filter_index(. ~ "2010-03-31")
atm1_test <- atm1_filled |> filter_index("2010-04-01" ~ .)

# Refit models on training data
atm1_fit_train <- atm1_train |>
  model(arima = ARIMA(Cash),
        snaive = SNAIVE(Cash ~ lag("week")),
        snaive_drift = SNAIVE(Cash ~ drift()),
        ets = ETS(Cash))

# Forecast on test data
atm1_fc <- atm1_fit_train |>
  forecast(h = "14 days")

# Plot forecasts against test data
atm1_fc |>
  autoplot(atm1_test) +
  labs(title = "ATM 1 Forecast vs Actuals",
       y = "Cash Balance",
       x = "Date")

```



```
# Check accuracy on test data
atm1_fc |>
  accuracy(bind_rows(atm1_train, atm1_test)) |>
  select(-all_of(exclude_metrics)) |>
  kable(caption="ATM 1 Test Accuracy")
```

Table 2: ATM 1 Test Accuracy

.model	ATM	.type	RMSE	MASE	ACF1
arima	ATM1	Test	10.845956	0.4466658	0.1660851
ets	ATM1	Test	9.706741	0.4073603	0.1684993
snaive	ATM1	Test	14.762404	0.6898781	0.3479893
snaive_drift	ATM1	Test	14.849069	0.6949759	0.3497395

ATM 2

We've taken a similar approach to ATM 2. We should note that we are using linear interpolation for missing values (June 17th 2009 and June 25th 2009) here instead of seasonal naive, as the gaps are small and infrequent, and we have reasonable surrounding observations that allow us to do so.

After imputing and fitting several models, we find ETS performs the best again, with ARIMA close behind.

Recommendation for ATM 2: Deploy an ETS model and monitor for drift over time.

- We'll perform similar steps for ATM 2
- Using `imputeTS` library going forward for missing dates
- Can't use MPE or MAPE due to zero levels for some observations
- ARIMA again has best point accuracy
- ETS and ARIMA perform well, but ETS performs slightly better and is simpler

```
# Check for gaps
scan_gaps(atm_data |> filter(ATM == "ATM2")) # should be empty
```

```
# A tibble: 2 x 2 [1D]
# Key:      ATM [1]
  ATM  DATE
  <chr> <date>
1 ATM2 2009-06-18
2 ATM2 2009-06-24
```

```
# Check surrounding dates
atm_data |>
  filter(ATM == "ATM2") |>
  filter_index("2009-06-17" ~ "2009-06-25") |>
  kable(caption="ATM 2 Around Missing Values")
```

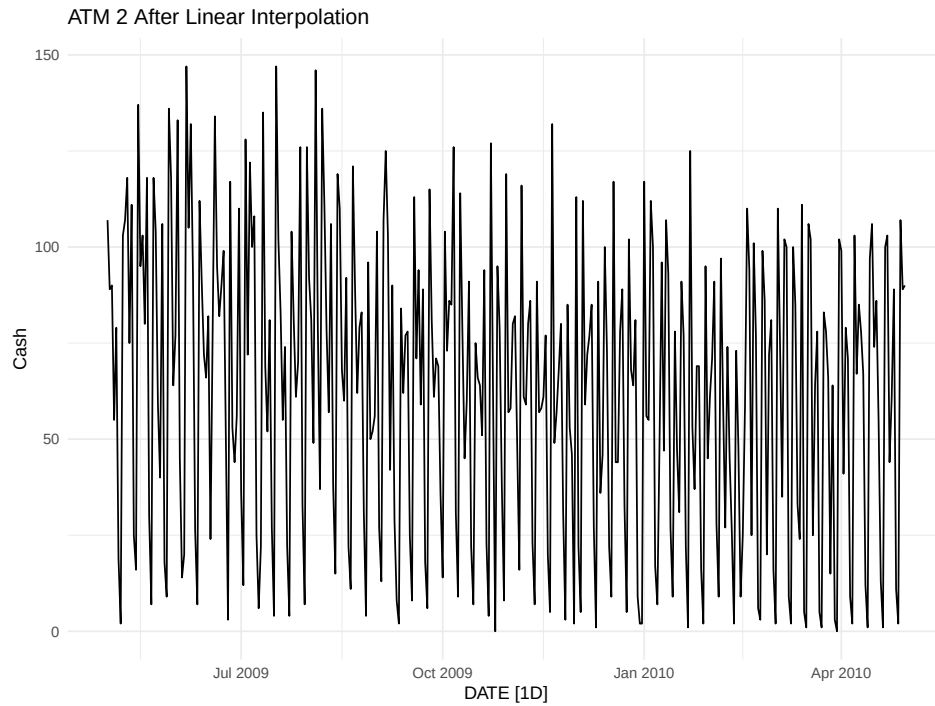
Table 3: ATM 2 Around Missing Values

DATE	ATM	Cash
2009-06-17	ATM2	24
2009-06-19	ATM2	134
2009-06-20	ATM2	95
2009-06-21	ATM2	82
2009-06-22	ATM2	90
2009-06-23	ATM2	99
2009-06-25	ATM2	3

```
atm_data <- atm_data |>
  group_by(ATM) |>
  fill_gaps() |>
  mutate(Cash = na_interpolation(Cash, option = "linear")) |>
  ungroup()
```

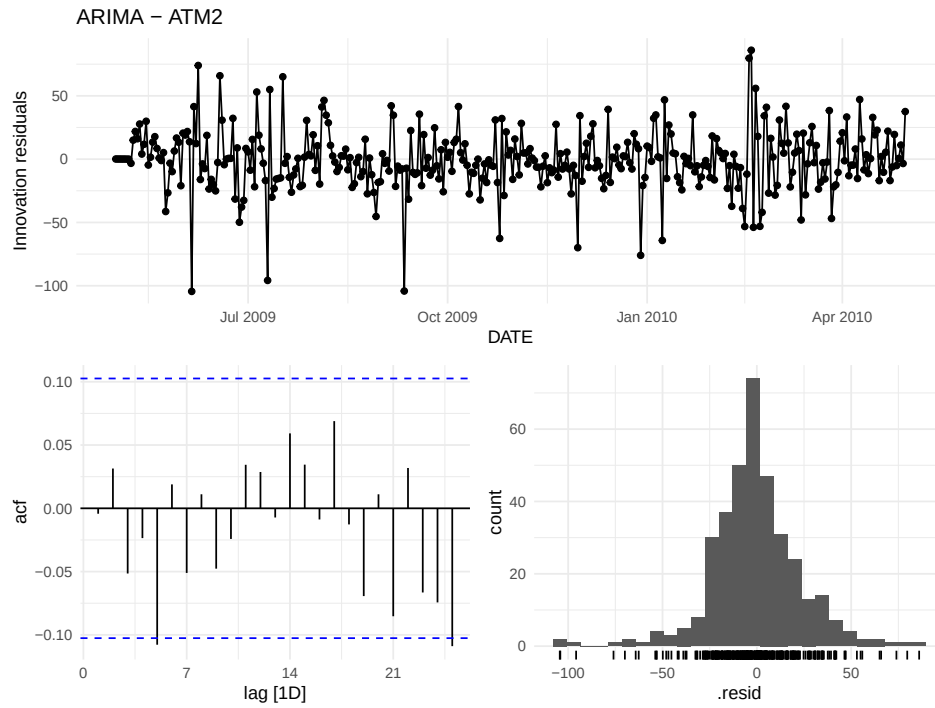
```
# Get ATM 2 data
atm2 <- atm_data |>
  filter(ATM == "ATM2")

autoplot(atm2) +
  labs(title="ATM 2 After Linear Interpolation")
```

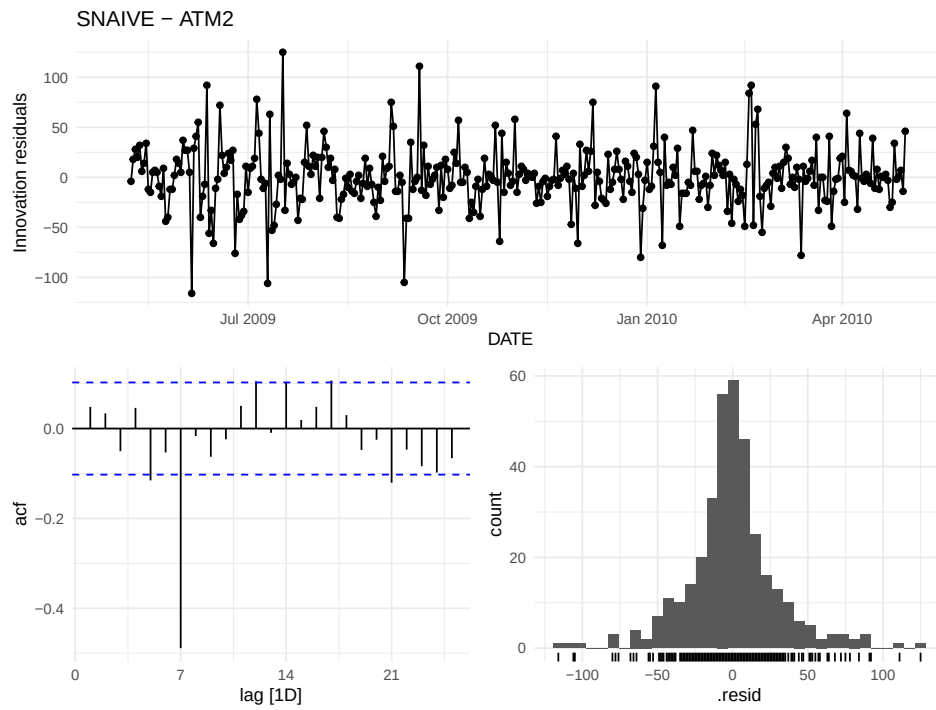


```
atm2_fit <- atm2 |>
  model(
    arima = ARIMA(Cash),
    snaive = SNAIVE(Cash ~ lag("week")),
    snaive_drift = SNAIVE(Cash ~ drift()),
    ets = ETS(Cash)
  )

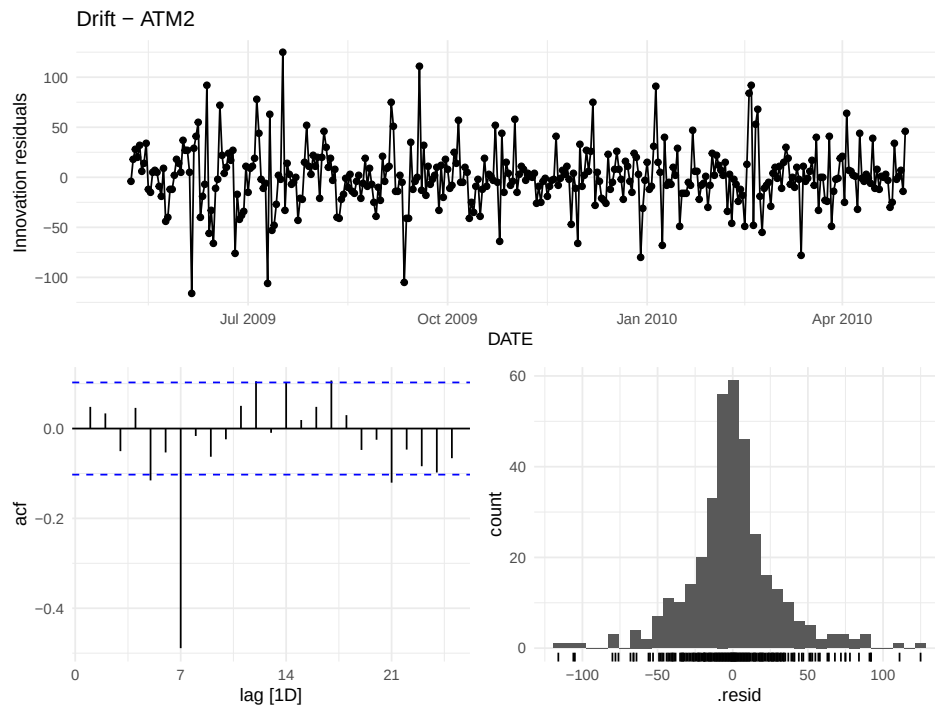
atm2_fit |>
  select(arima) |>
  gg_tsresiduals() + labs(title = "ARIMA - ATM2")
```



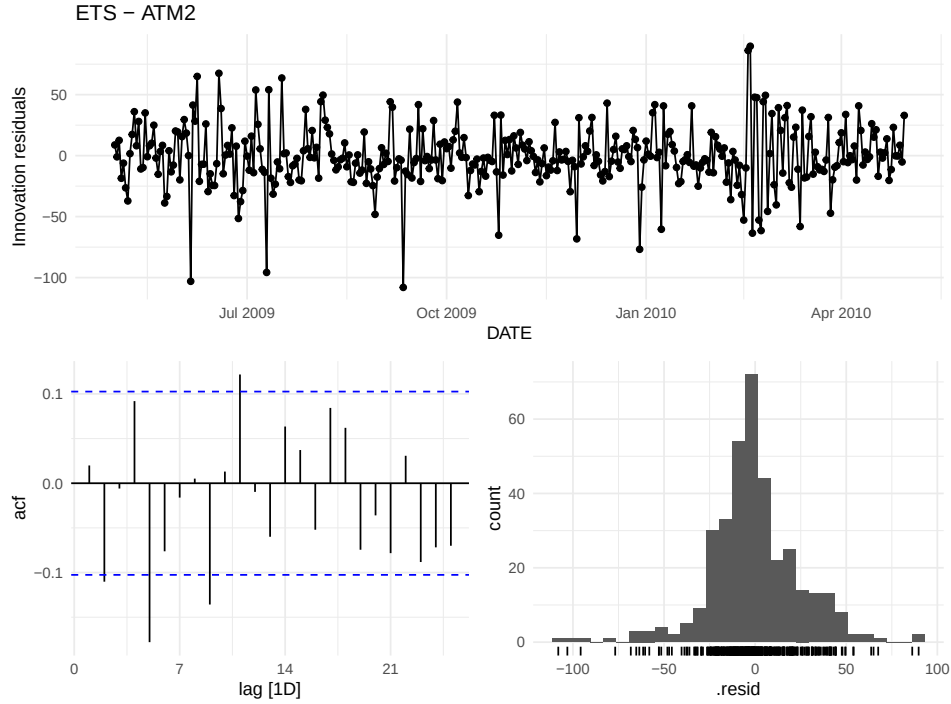
```
atm2_fit |>
  select(snaive) |>
  gg_tsresiduals() + labs(title = "SNAIVE - ATM2")
```



```
atm2_fit |>
  select(snaive_drift) |>
  gg_tsresiduals() + labs(title = "Drift - ATM2")
```



```
atm2_fit |>
  select(ets) |>
  gg_tsresiduals() + labs(title = "ETS - ATM2")
```

```
atm2_fit |>
  accuracy() |>
  select(-all_of(exclude_metrics)) |>
  kable(caption="ATM 2 Point Estimate Accuracy")
```

Table 4: ATM 2 Point Estimate Accuracy

ATM	.model	.type	RMSE	MASE	ACF1
ATM2	arima	Training	24.13932	0.8199456	-0.0042753
ATM2	snaive	Training	30.19721	1.0000000	0.0481966
ATM2	snaive_drift	Training	30.19720	1.0000661	0.0481966
ATM2	ets	Training	25.14201	0.8594547	0.0198839

```
# Split the data into train and test
atm2_train <- atm2 |> filter_index(. ~ "2010-03-31")
atm2_test <- atm2 |> filter_index("2010-04-01" ~ .)

# Refit models on training data
atm2_fit_train <- atm2_train |>
  model(arima = ARIMA(Cash),
```

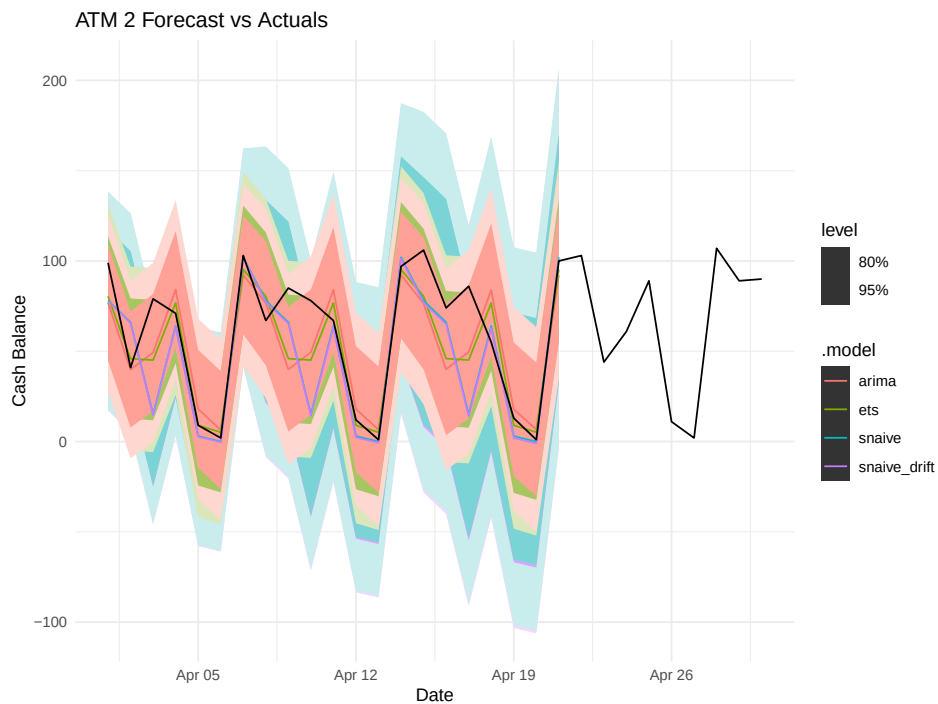
```

snaive = SNAIVE(Cash ~ lag("week")),
snaive_drift = SNAIVE(Cash ~ drift()),
ets = ETS(Cash))

# Forecast on test data
atm2_fc <- atm2_fit_train |>
  forecast(h = "21 days")

# Plot forecasts against test data
atm2_fc |>
  autoplot(atm2_test) +
  labs(title = "ATM 2 Forecast vs Actuals",
       y = "Cash Balance",
       x = "Date")

```



```

# Check accuracy on test data
atm2_fc |>
  accuracy(bind_rows(atm2_train, atm2_test)) |>
  select(-all_of(exclude_metrics)) |>
  kable(caption="ATM 2 Test Accuracy")

```

Table 5: ATM 2 Test Accuracy

.model	ATM	.type	RMSE	MASE	ACF1
arima	ATM2	Test	21.13362	0.7912142	0.0905425
ets	ATM2	Test	19.73485	0.6870531	-0.0888763
snaive	ATM2	Test	27.51104	0.8179384	-0.2915089
snaive_drift	ATM2	Test	27.84997	0.8333367	-0.2891225

ATM 3

ATM 3 is unique in that it appears to be a new machine, or perhaps recently reinstated, given the data has zero balances until the last three days of the series.

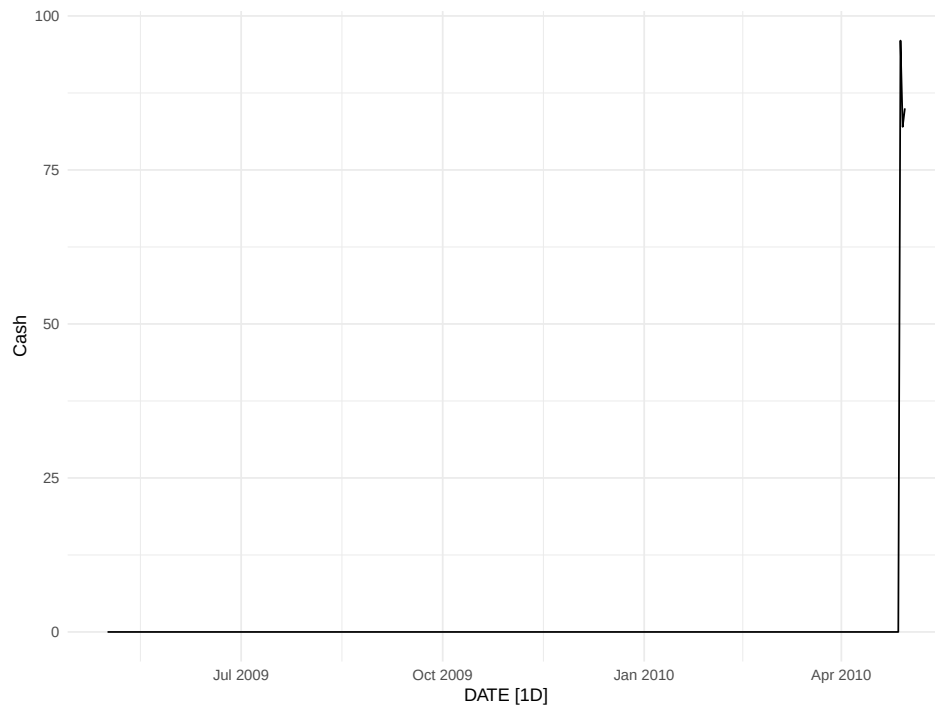
With such limited data, we really can't develop a proper model. From a pure business perspective, we could use the domain knowledge we have about the machine to find a similar machine with more data, and use that to guide our forecasts.

Until additional data is available, we will simply use a naive random walk to model future balances.

Recommendation for ATM 3: Source data for similar machine and model. Alternatively, deploy naive random walk until sufficient data to train a more robust model.

```
# Get ATM 3 data
atm3 <- atm_data |>
  filter(ATM == "ATM3")

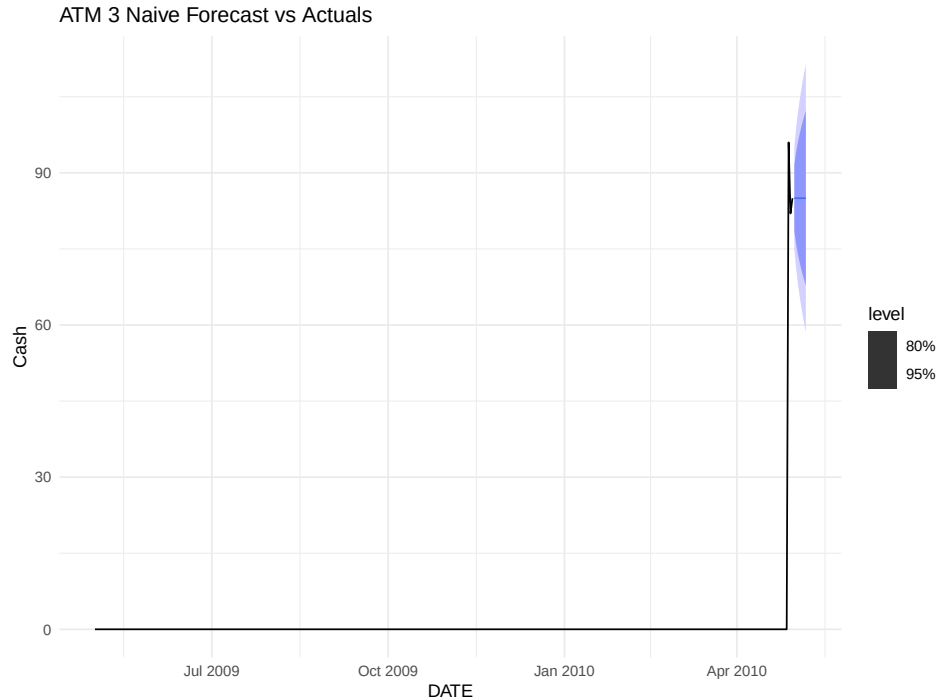
autoplot(atm3)
```



```
atm3_fit <- atm3 |>
  model(RW(Cash))

# Forecast on test data
atm3_fc <- atm3_fit |>
  forecast(h = "7 days")

atm3_fc |>
  autoplot(atm3) +
  labs(title = "ATM 3 Naive Forecast vs Actuals")
```



ATM 4

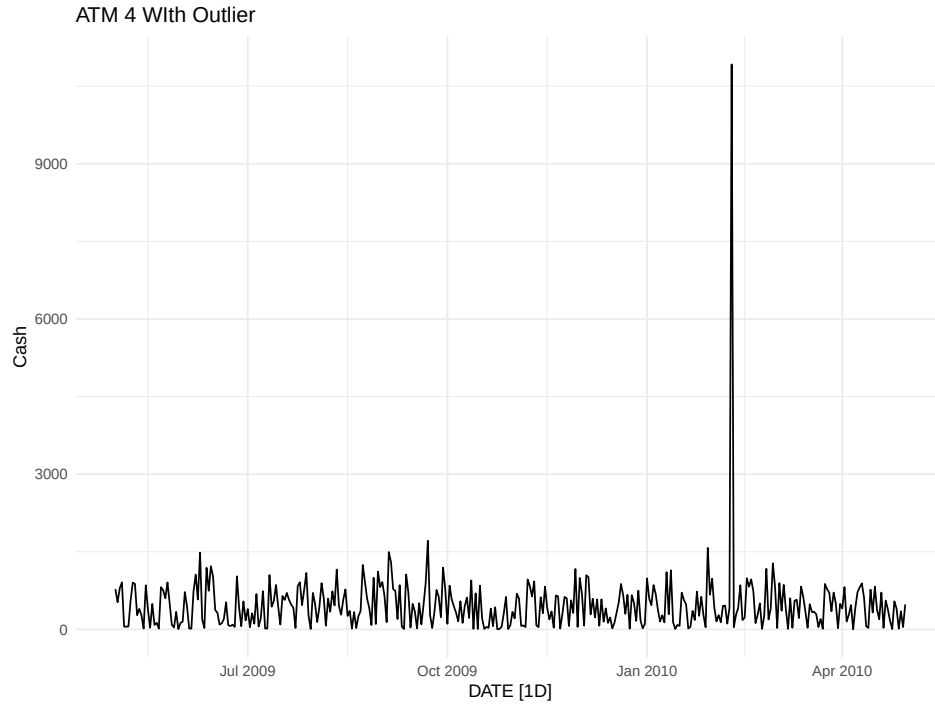
We see an outlier that is orders of magnitude larger than the rest of the ATM data collectively. This value is likely erroneous, as it is unlikely that a single ATM would hold such a large cash balance.

Again, since we have surrounding data, we will linearly impute the outlier and proceed with modeling.

Seasonal naive, both with and without drift, performed very well with this series, the drift constant slightly improving MASE scores. As a high-performing simple model, it is a straight-forward choice. We recommend the drift model to allow for better long-term forecasting in case any trend begins to appear.

Recommendation for ATM 4: Deploy a seasonal naive model with drift

```
# Get ATM 4 data
atm4 <- atm_data |>
  filter(ATM == "ATM4")
autoplot(atm4) +
  labs(title="ATM 4 WItH Outlier")
```



```
# Find the outlier
atm4 |>
  slice_max(Cash, n=1)
```

```
# A tsibble: 1 x 3 [1D]
# Key:      ATM [1]
  DATE      ATM      Cash
  <date>    <chr>  <dbl>
1 2010-02-09 ATM4   10920.
```

```
# Check around the observation
atm4 |>
  filter_index("2010-02-07" ~ "2010-02-12") |>
  kable(caption="ATM 4 Outlier on 2010-02-09")
```

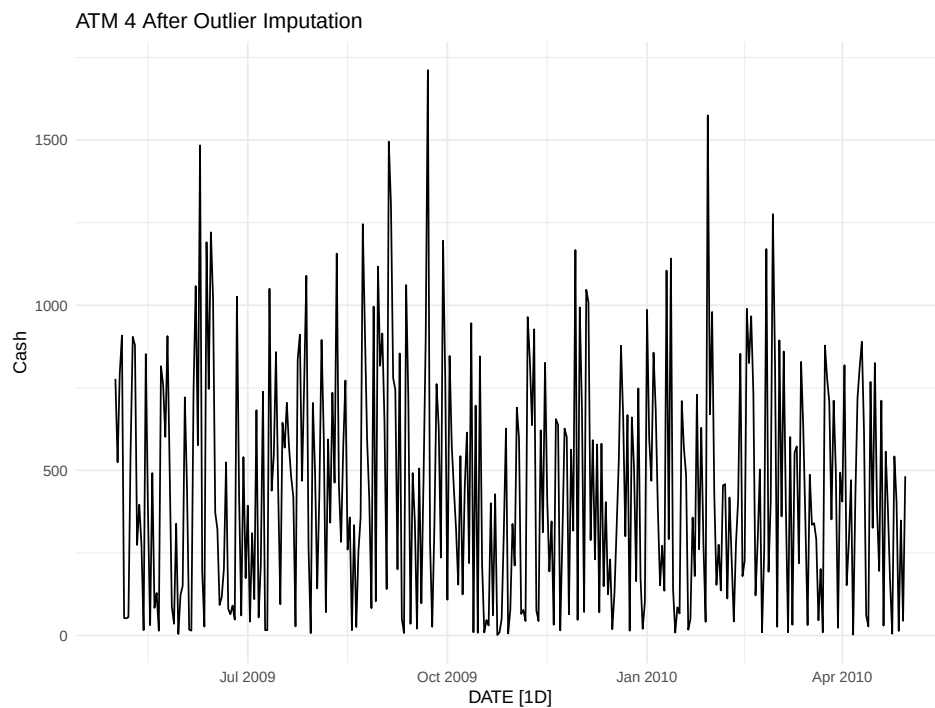
Table 6: ATM 4 Outlier on 2010-02-09

DATE	ATM	Cash
2010-02-07	ATM4	112.03063

DATE	ATM	Cash
2010-02-08	ATM4	417.91228
2010-02-09	ATM4	10919.76164
2010-02-10	ATM4	42.43808
2010-02-11	ATM4	280.04343
2010-02-12	ATM4	412.31888

```
# Impute the outlier using linear interpolation
atm4 <- atm4 |>
  mutate(Cash = if_else(DATE == as.Date("2010-02-09"), NA_real_, Cash)) |>
  mutate(Cash = na_interpolation(Cash, option = "linear"))

# Check
autoplot(atm4) +
  labs(title="ATM 4 After Outlier Imputation")
```



```
# Split data
atm4_train <- atm4 |> filter_index(. ~ "2010-03-31")
atm4_test <- atm4 |> filter_index("2010-04-01" ~ .)
```

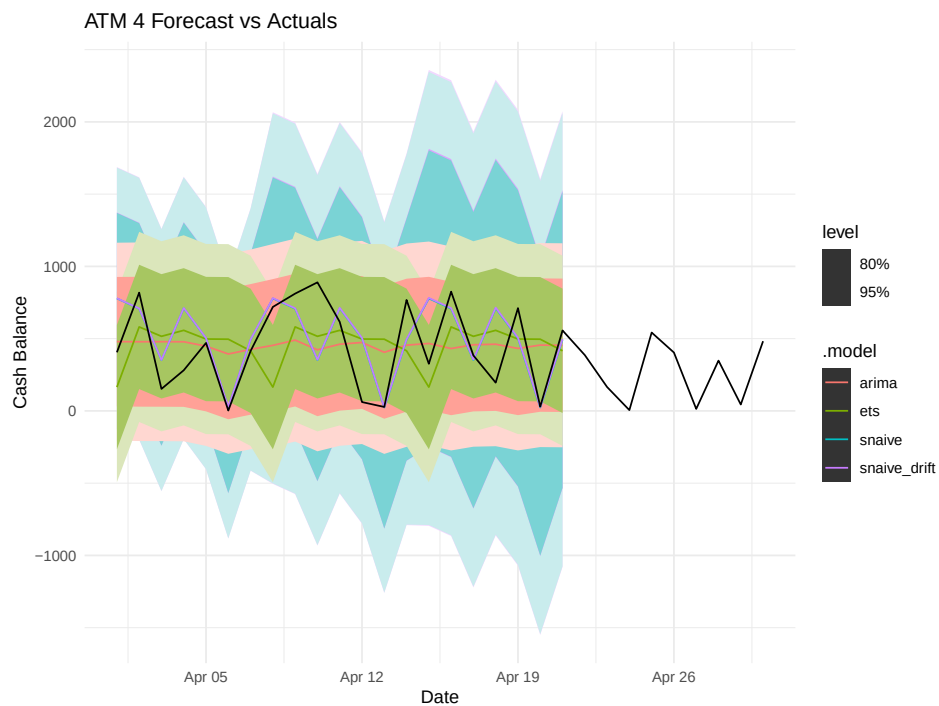
```

# Fit models on training data
atm4_fit_train <- atm4_train |>
  model(
    arima = ARIMA(Cash),
    snaive = SNAIVE(Cash ~ lag("week")),
    snaive_drift = SNAIVE(Cash ~ drift()),
    ets = ETS(Cash)
  )

# Forecast on test data
atm4_fc <- atm4_fit_train |>
  forecast(h = "21 days")

# Plot forecasts against test data
atm4_fc |>
  autoplot(atm4_test) +
  labs(
    title = "ATM 4 Forecast vs Actuals",
    y = "Cash Balance",
    x = "Date"
  )

```




```
# Check accuracy on test data
atm4_fc |>
  accuracy(bind_rows(atm4_train, atm4_test)) |>
  select(-all_of(exclude_metrics)) |>
  kable(caption = "ATM 4 Test Accuracy")
```

Table 7: ATM 4 Test Accuracy

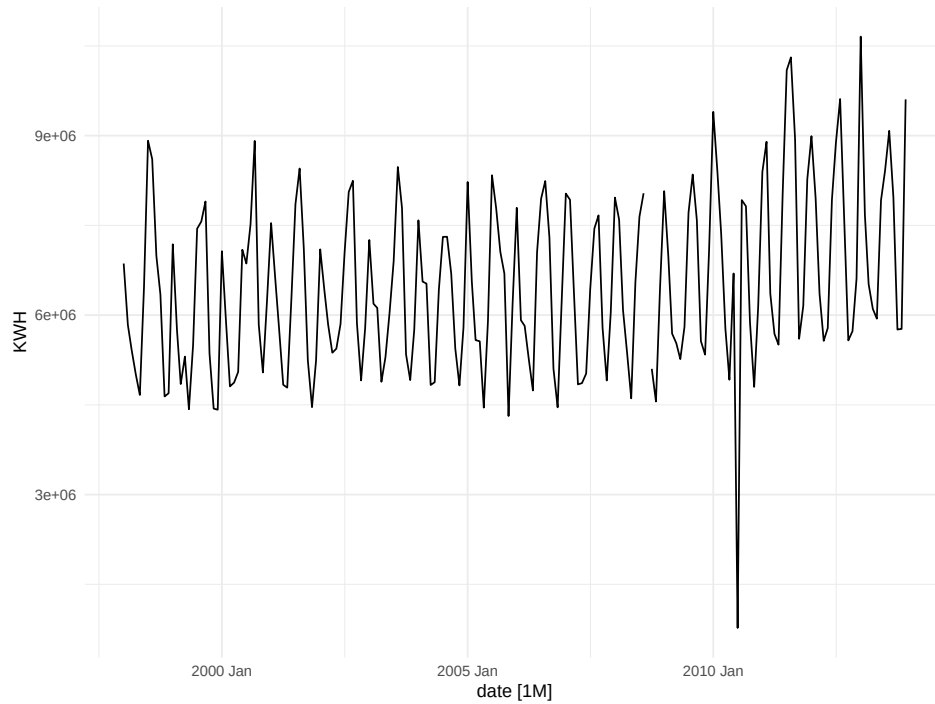
.model	ATM	.type	RMSE	MASE	ACF1
arima	ATM4	Test	290.1291	0.7226830	-0.0832433
ets	ATM4	Test	317.9215	0.7873436	0.0965703
snaive	ATM4	Test	267.1955	0.5608140	-0.1899113
snaive_drift	ATM4	Test	267.6443	0.5598168	-0.1911242

Part B - Forecasting Power

- The series is relatively consistent except for two outliers
- One outlier is an order of magnitude lower than the rest of the data
 - We can simply raise it by a factor of 10 to impute it
- The other is simply missing
 - Visually it is similar to previous season
 - We should use seasonal imputation to fill it
- We impute for model training, but should note it in the analysis

```
# Read in power data
power <- readxl::read_excel("ResidentialCustomerForecastLoad-624.xlsx") |>
  mutate(date = yearmonth(`YYYY-MMM`)) |>
  select(-`YYYY-MMM`) |>
  as_tsibble(index=date)

autoplot(power, KWH)
```



```
# Check for gaps in series
scan_gaps(power) # should be empty
```

```
# A tsibble: 0 x 1 [?]
# i 1 variable: date <mth>
```

```
# Check for null values
power |>
  filter(is.na(KWH)) # 2008 Sep is missing
```

```
# A tsibble: 1 x 3 [1M]
  CaseSequence  KWH    date
      <dbl> <dbl>   <mth>
1         861    NA 2008 Sep
```

```
# Check the min outlier
power |>
  slice_min(KWH, n=1)
```

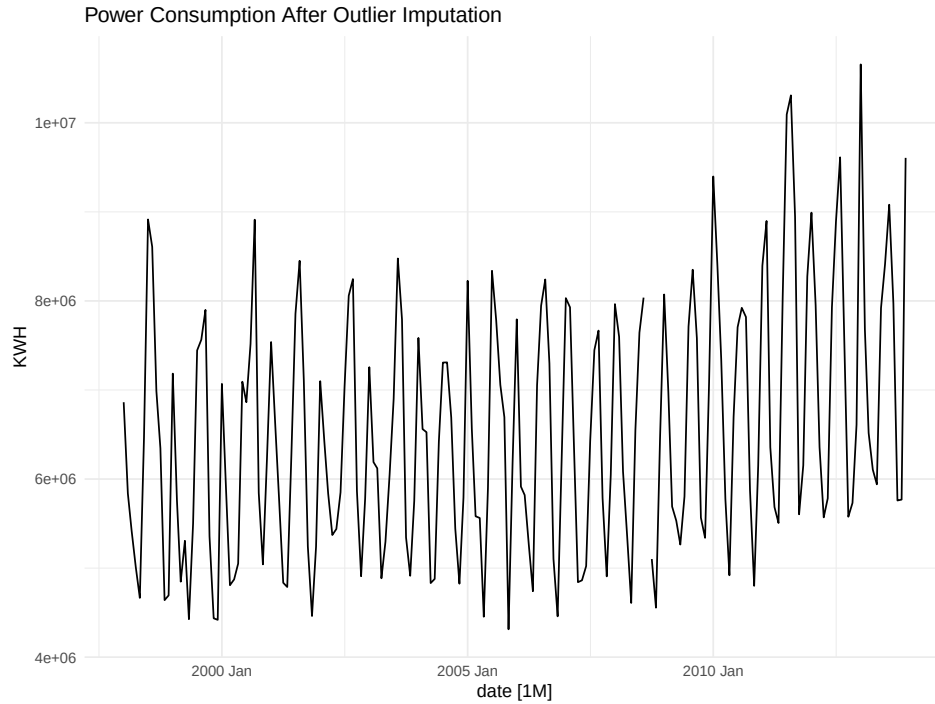
```
# A tsibble: 1 x 3 [1M]
  CaseSequence    KWH    date
      <dbl>  <dbl>  <mth>
1         883 770523 2010 Jul
```

```
# Check around the outlier on July 2010
power |>
  filter_index("2010-06" ~ "2010-08") |>
  kable(caption="Power Consumption Around July 2010 Outlier")
```

Table 8: Power Consumption Around July 2010 Outlier

CaseSequence	KWH	date
882	6696292	2010 Jun
883	770523	2010 Jul
884	7922701	2010 Aug

```
# Impute low outlier by raising it by a factor of 10
power_imp <- power |>
  mutate(KWH = if_else(date == yearmonth("2010 Jul"), KWH * 10, KWH))
autoplot(power_imp, KWH) +
  labs(title="Power Consumption After Outlier Imputation")
```



```
# Impute missing outlier using seasonal naive lag = 12
power_imp <- power_imp |>
  fill_gaps() |>
  mutate(KWH = if_else(is.na(KWH), lag(KWH, 12), KWH))

# Check September 2008 and surrounding obs
power_imp |>
  filter(date %in% c(
    yearmonth("2007 Sep"),
    yearmonth("2006 Sep"),
    yearmonth("2009 Sep")
  )) |>
  kable(caption = "Power Consumption After Missing Value Imputation")
```

Table 9: Power Consumption After Missing Value Imputation

CaseSequence	KWH	date
837	7296355	2006 Sep
849	7666970	2007 Sep
873	7583146	2009 Sep

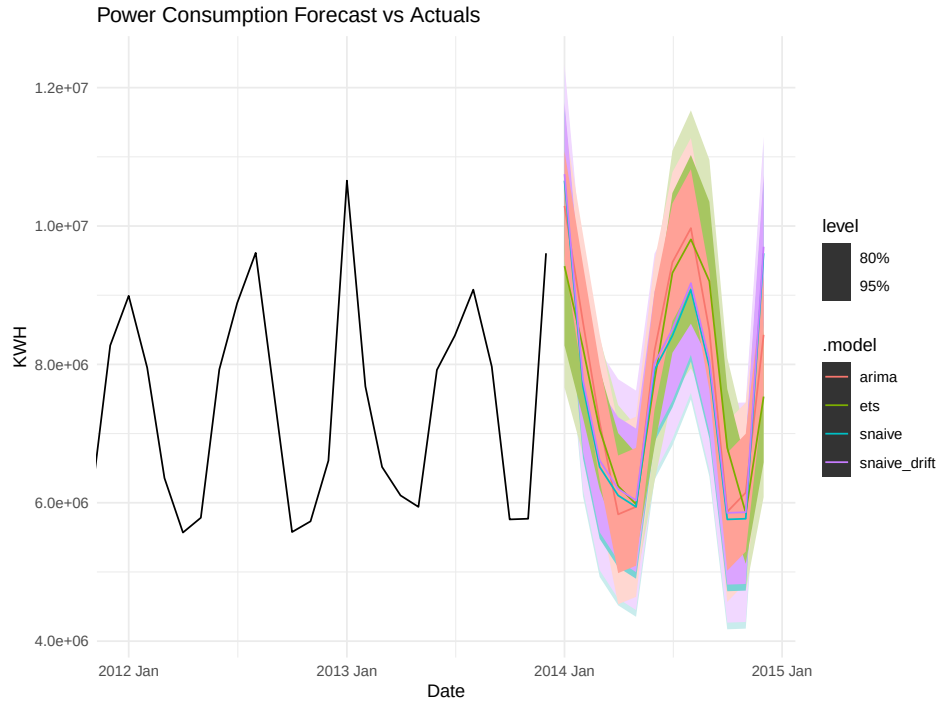
- With the outliers handled, we can proceed to modeling
- Very seasonal data so we will try a few seasonal models
- Slight trend upwards overtime

```
power_fit <- power_imp |>
  model(
    arima = ARIMA(KWH),
    snaive = SNAIVE(KWH ~ lag("year")),
    snaive_drift = SNAIVE(KWH ~ drift()),
    ets = ETS(KWH)
  )

# Forecast through 2014

power_fc <- power_fit |>
  forecast(h = 12)

# Visually Check
power_fc |>
  autoplot(power_imp) +
  coord_cartesian(xlim = c(yearmonth("2012 Jan"), yearmonth("2014 Dec"))) +
  labs(title = "Power Consumption Forecast vs Actuals",
       y = "KWH",
       x = "Date")
```



- Visually all of these models seem to fit well
- Let's check the accuracy metrics to choose the best one

```
power_fit |>
  accuracy() |>
  select(-all_of(exclude_metrics)) |>
  kable(caption="Power Consumption Point Estimate Accuracy")
```

Table 10: Power Consumption Point Estimate Accuracy

.model	.type	RMSE	MASE	ACF1
arima	Training	584708.3	0.6947968	0.0015633
snaive	Training	810311.8	1.0000000	0.2904251
snaive_drift	Training	804812.4	1.0005092	0.2904251
ets	Training	613840.6	0.7485217	0.2855910

```
power_fit |>
  select(arima)
```

```
# A mable: 1 x 1
```

```

              arima
              <model>
1 <ARIMA(0,0,4)(2,1,0)[12] w/ drift>

```

- ARIMA with (0,0,4) and seasonal component (2,1,0) with drift has been chosen
- This seems reasonable; stationary series, very little autoregressive behavior, but some moving average components to smooth out noise
 - The drift address the slightly upwards trend over time
 - The seasonal component captures the yearly seasonality
- Let's test these models on a train/test split to see if they hold up

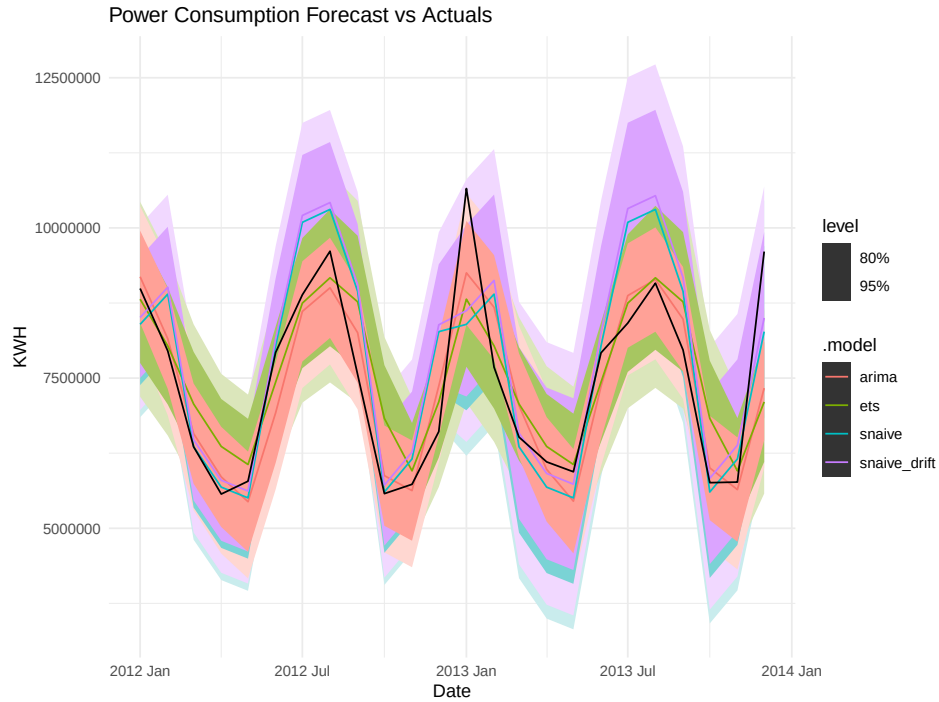
```

# Split the data into train and test
power_train <- power_imp |> filter_index(. ~ "2011 Dec")
power_test <- power_imp |> filter_index("2012 Jan" ~ .)
# Refit models on training data
power_fit_train <- power_train |>
  model(
    arima = ARIMA(KWH),
    snaive = SNAIVE(KWH ~ lag("year")),
    snaive_drift = SNAIVE(KWH ~ drift()),
    ets = ETS(KWH),
  )

# Forecast on test data
power_fc <- power_fit_train |>
  forecast(h = 24)

# Plot forecasts against test data
power_fc |> autoplot(power_test) +
  labs(title = "Power Consumption Forecast vs Actuals",
       y = "KWH",
       x = "Date")

```



```
# Check accuracy
power_fc |>
  accuracy(bind_rows(power_train, power_test)) |>
  select(-all_of(exclude_metrics)) |>
  kable(caption="Power Consumption Test Accuracy")
```

Table 11: Power Consumption Test Accuracy

.model	.type	RMSE	MASE	ACF1
arima	Test	721020.6	0.8771826	-0.0782669
ets	Test	851010.3	1.0231905	0.0491969
snaive	Test	965567.1	1.2231527	-0.1987993
snaive_drift	Test	1017311.3	1.3051110	-0.2033422

- ARIMA still holds out as the best model on the test data
- We would only wish to use another model for simplicity
- Again, we should note the imputation steps taken in the analysis

Part C - Waterflow Pipes

Part C consists of two data sets. These are simple 2 columns sets, however they have different time stamps. Your optional assignment is to time-base sequence the data and aggregate based on hour (example of what this looks like, follows). Note for multiple recordings within an hour, take the mean. Then to determine if the data is stationary and can it be forecast. If so, provide a week forward forecast and present results via Rpubs and .rmd and the forecast in an Excel readable file.

I took the below steps to model this series;

- We replace multiple recordings within an hour by the mean
- We aggregate the data, effectively extending the series
- Observe Data is not stationary
- BoxCox has small effect but does normalize the variance, so we will use it
- The series is made stationary with a seasonal difference of lag 2 based on the two day correlation we saw
- We now model on the preprocessed data as a train / test split
- Checking the residuals, they are white noise
- Proceed to forecast
- Calculate accuracy metrics

By spending time preprocessing the data to make it stationary, we were able to train an ARIMA model to forecast the next seven days with a MASE of 0.5229456, which is decent performance for a limited training set.

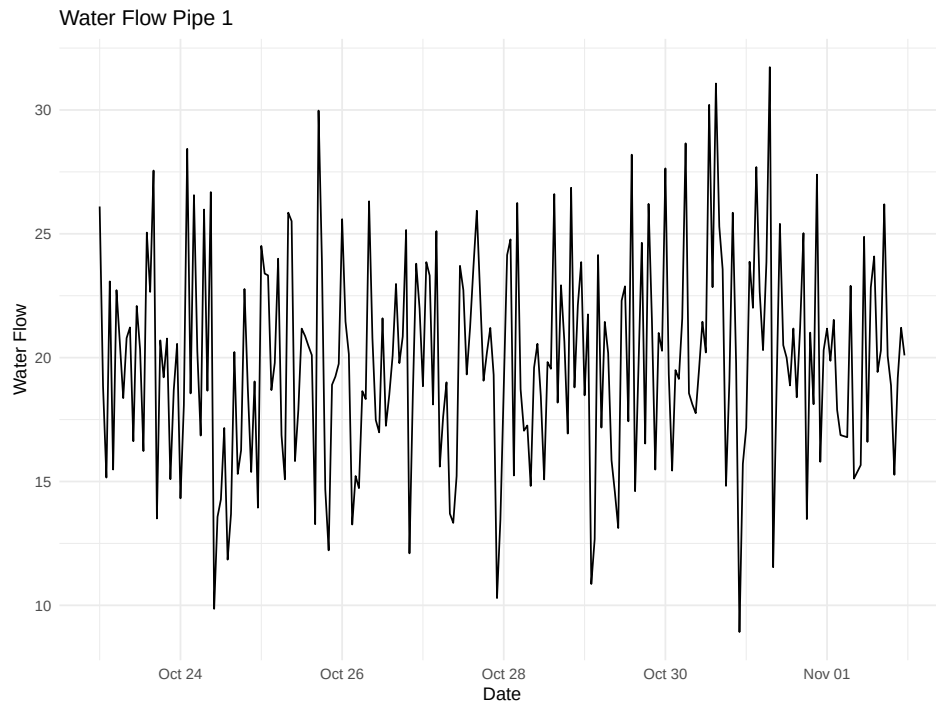
```
pipe1 <- readxl::read_excel("Waterflow_Pipe1.xlsx") |>
  mutate(datetime = as.POSIXct(`Date Time` * 86400, origin = "1899-12-30", tz = "UTC"),
         datetime_hour = floor_date(datetime, "hour")) |>
  group_by(datetime_hour) |>
  summarize(WaterFlow = mean(WaterFlow, na.rm = TRUE))

pipe1_ts <- pipe1 |>
  as_tsibble(index = datetime_hour)

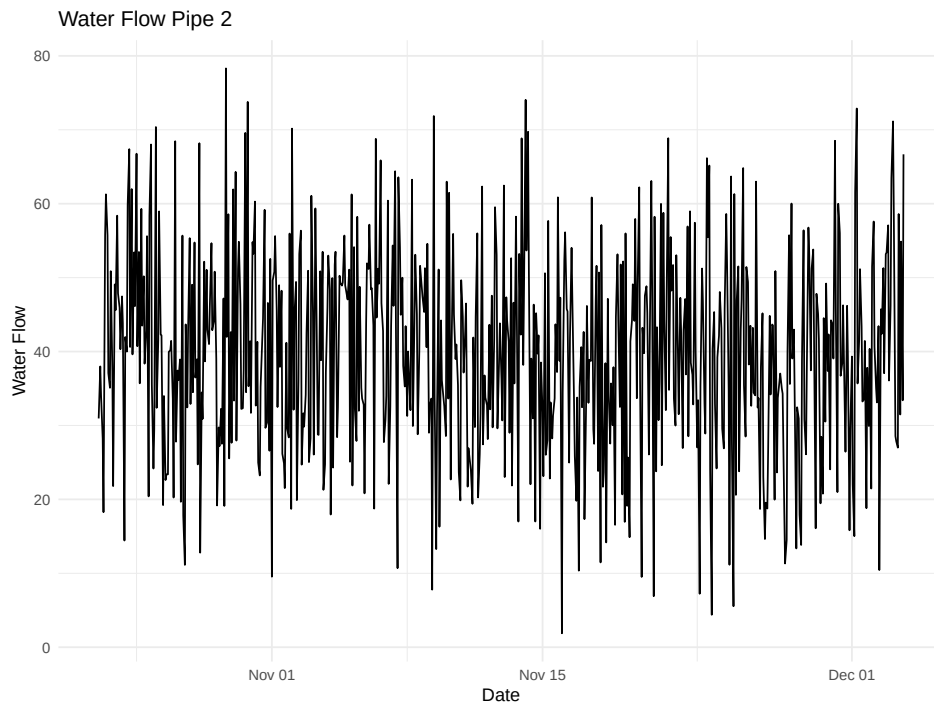
pipe2 <- readxl::read_excel("Waterflow_Pipe2.xlsx") |>
  mutate(datetime = as.POSIXct(`Date Time` * 86400, origin = "1899-12-30", tz = "UTC"),
         datetime_hour = floor_date(datetime, "hour")) |>
  group_by(datetime_hour) |>
  summarize(WaterFlow = mean(WaterFlow, na.rm = TRUE))

pipe2_ts <- pipe2 |>
  as_tsibble(index = datetime_hour)
```

```
autoplot(pipe1_ts, WaterFlow) +  
  labs(title = "Water Flow Pipe 1", y = "Water Flow", x = "Date")
```

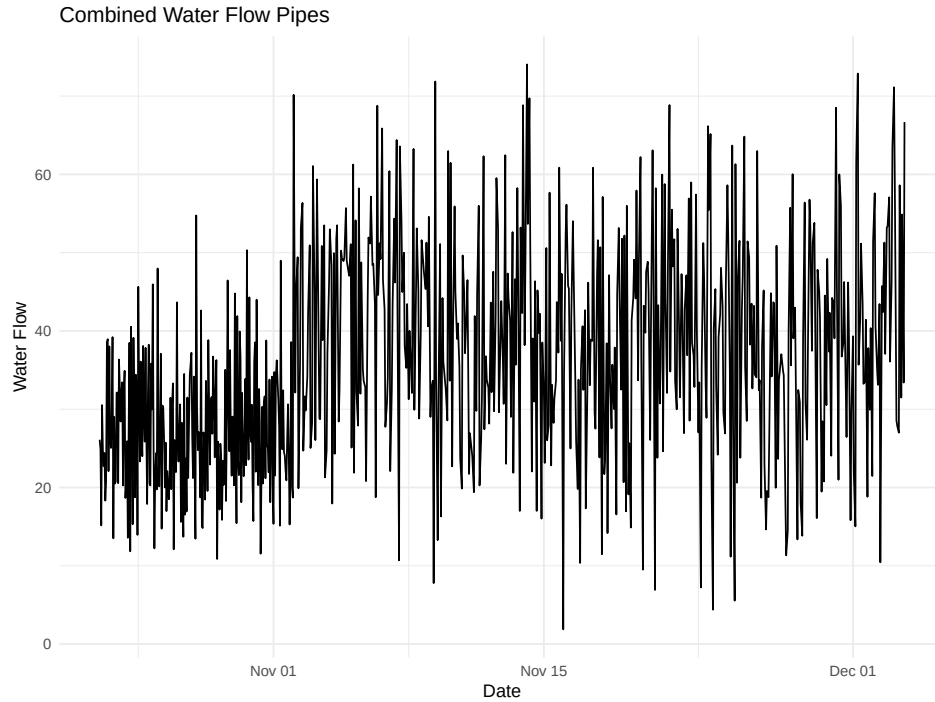


```
autoplot(pipe2_ts, WaterFlow) +  
  labs(title = "Water Flow Pipe 2", y = "Water Flow", x = "Date")
```



```
# Combine the data, taking the mean if there's overlap
pipes_combined <- full_join(pipe1_ts, pipe2_ts, by = "datetime_hour", suffix = c("_pipe1", ".
  rowwise() |>
  mutate(WaterFlow = mean(c(WaterFlow_pipe1, WaterFlow_pipe2), na.rm = TRUE)) |>
  ungroup() |>
  select(datetime_hour, WaterFlow) |>
  as_tsibble(index = datetime_hour)

# Plot the series
autoplot(pipes_combined, WaterFlow) +
  labs(title = "Combined Water Flow Pipes", y = "Water Flow", x = "Date")
```



We have a slew of missing values to deal with. Given they are generally not consecutive, a linear interpolation to connect the values of the last observation with the missing one seems to be reasonable.

```
pipes_combined |> scan_gaps()
```

```
# A tsibble: 255 x 1 [1h] <UTC>
  datetime_hour
  <dtm>
1 2015-10-27 17:00:00
2 2015-11-01 05:00:00
3 2015-11-02 02:00:00
4 2015-11-02 05:00:00
5 2015-11-02 08:00:00
6 2015-11-02 11:00:00
7 2015-11-02 14:00:00
8 2015-11-02 17:00:00
9 2015-11-02 20:00:00
10 2015-11-02 23:00:00
# i 245 more rows
```

```

pipes_combined <- pipes_combined |>
  fill_gaps() |>
  mutate(WaterFlow = na_interpolation(WaterFlow, option = "linear"))

```

With gaps filled, we must now adjust the variance in the time series using a BoxCox transformation

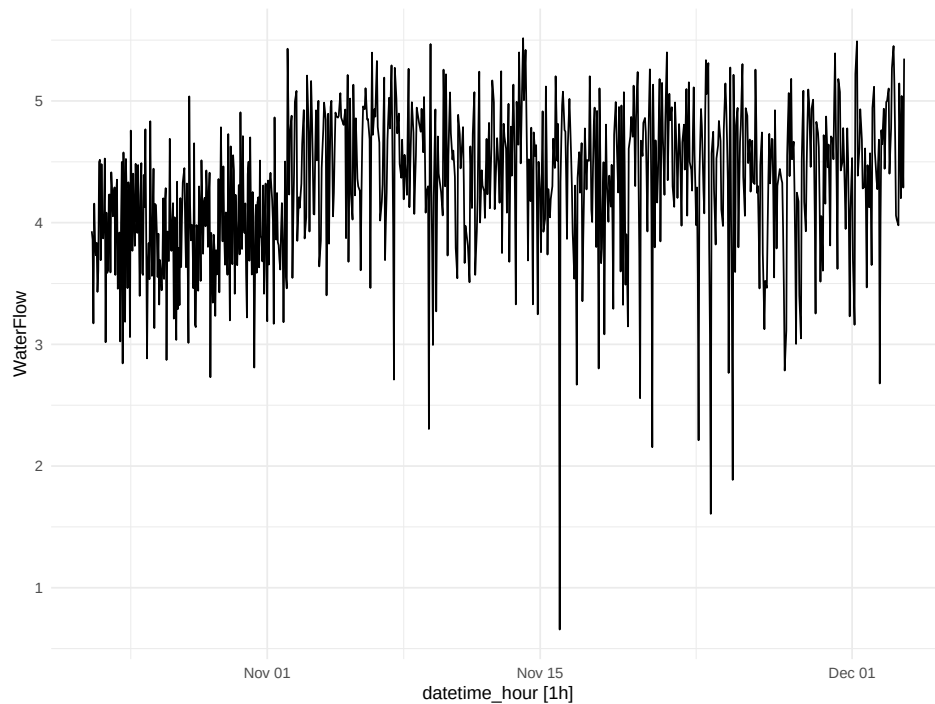
```

lambda <- pipes_combined |>
  features(WaterFlow, features = guerrero) |>
  pull(lambda_guerrero)

pipes_transformed <- pipes_combined |>
  mutate(WaterFlow = box_cox(WaterFlow, lambda))

autoplot(pipes_transformed)

```



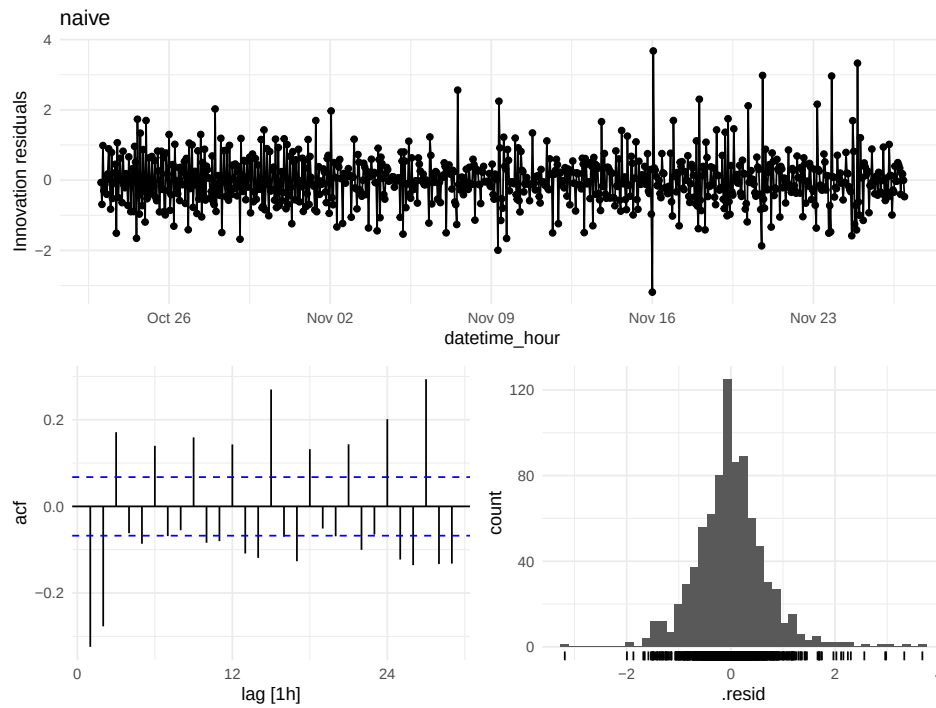
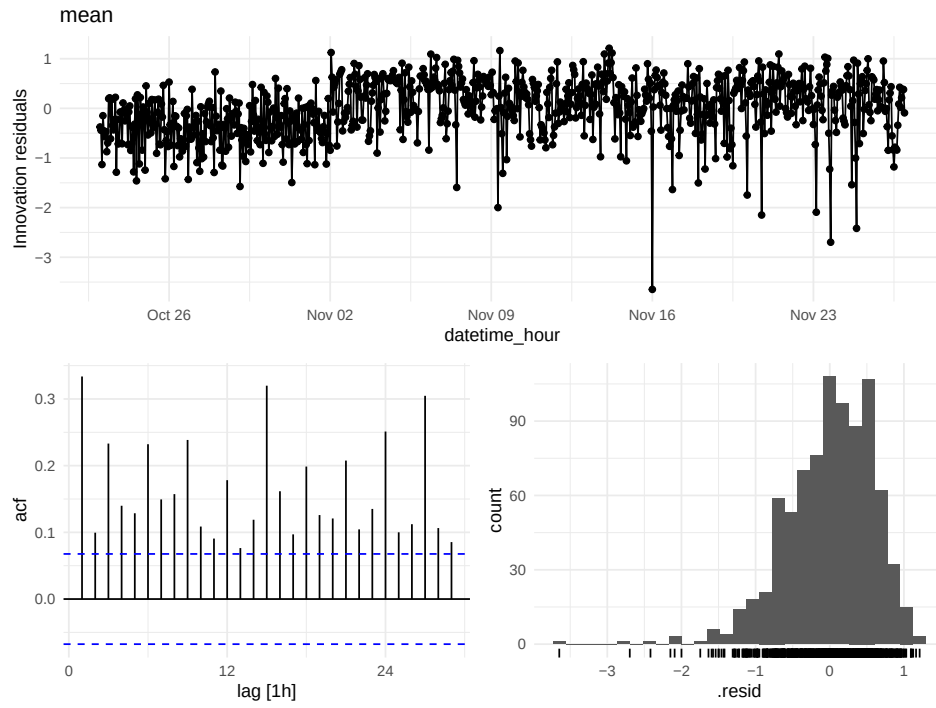
We split the data into test and train sets

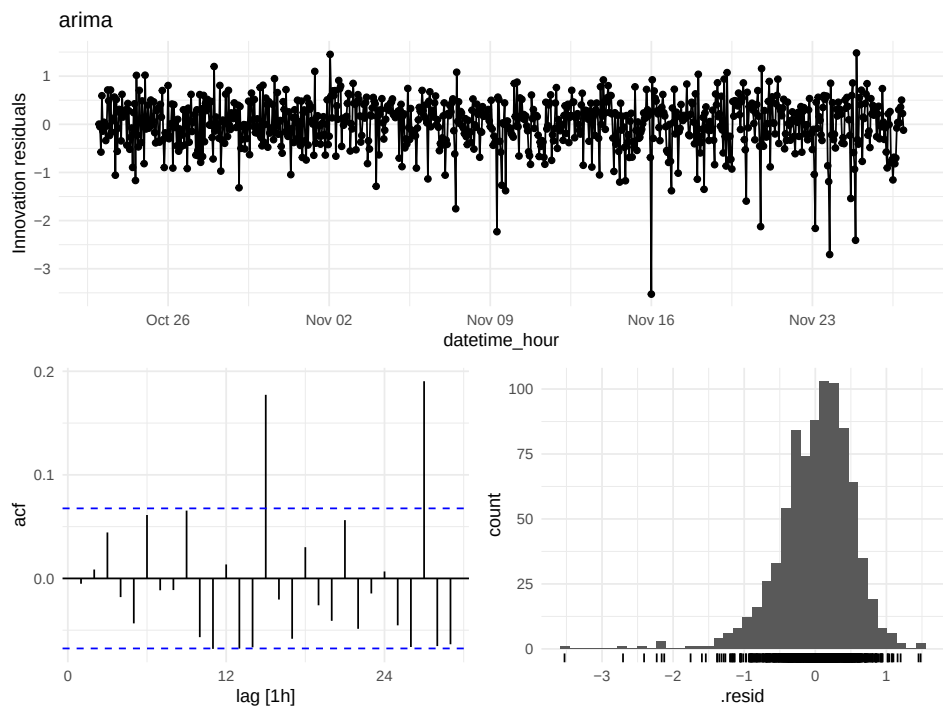
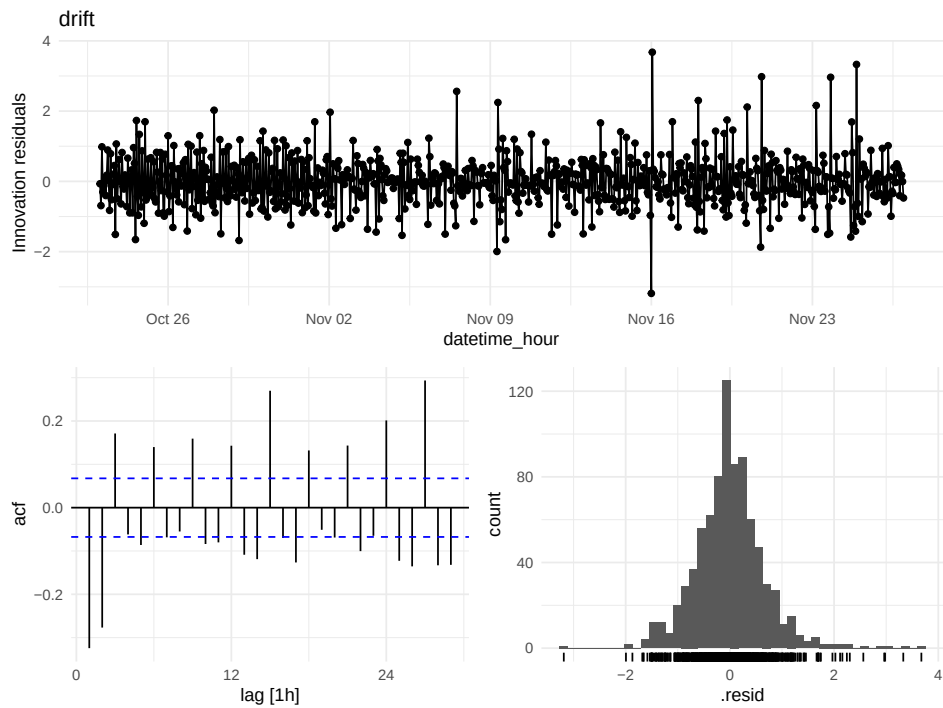
```
# Train test split
pipe_train <- pipes_transformed |> filter_index(~ "2015-11-26")
pipe_test <- pipes_transformed |> filter_index("2015-11-27" ~ .) |> drop_na()
```

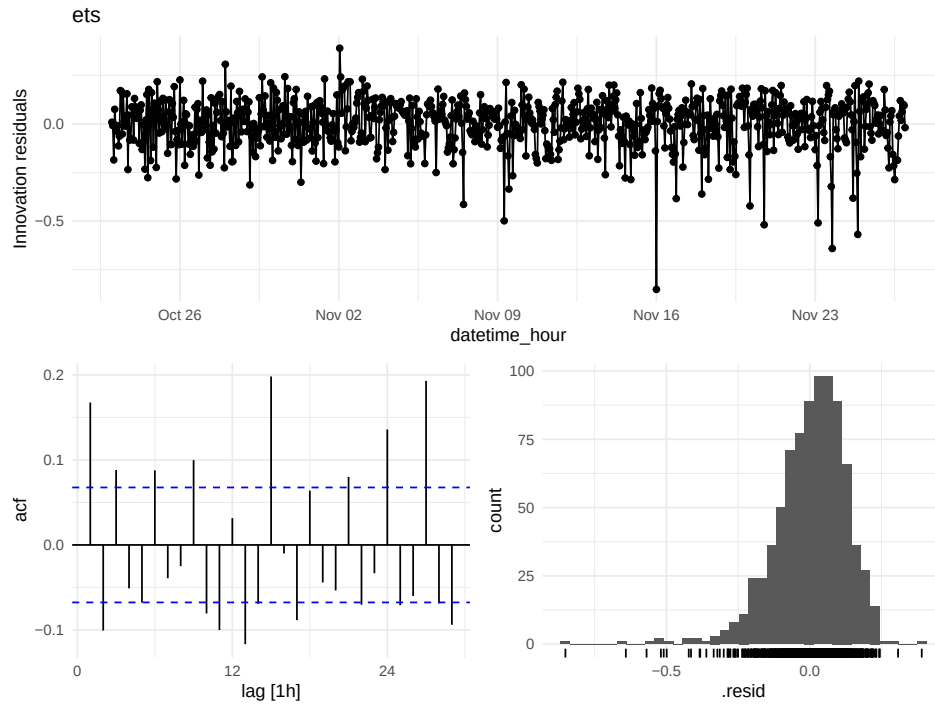
Now we train the model, checking residuals to make sure they visually look like white noise with no discernable patterns.

```
# Train models
pipes_fit <- pipe_train |>
  model(
    mean = MEAN(),
    naive = NAIVE(),
    drift = RW(~ drift()),
    arima = ARIMA(),
    ets = ETS()
  )

# Check residuals
for (model_name in names(pipes_fit)) {
  print(pipes_fit |> select(all_of(model_name)) |> gg_tsresiduals() +
    ggtitle(model_name))
}
```

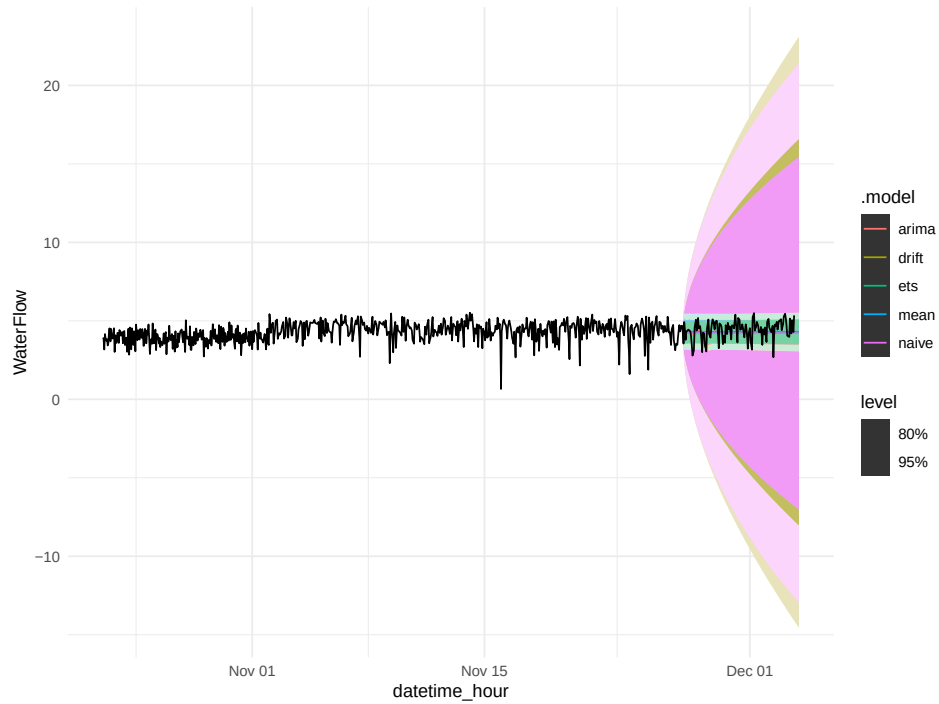






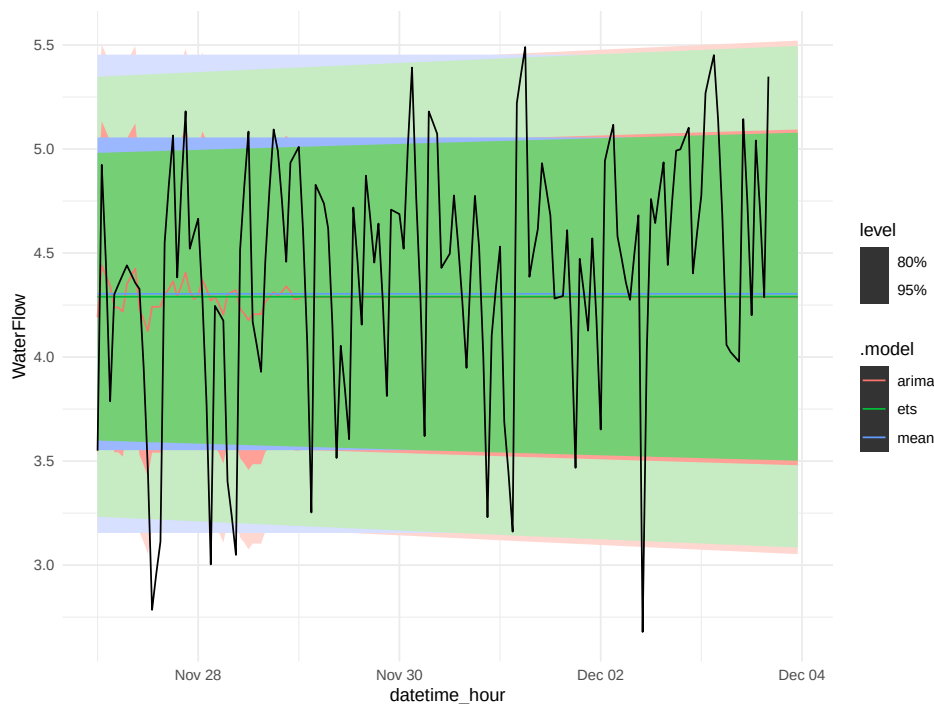
```
# Forecast on test data
pipes_fc <- pipes_fit |>
  forecast(h = "7 days")

# Plot forecasts against test data
pipes_fc |>
  autoplot(pipes_transformed)
```



Clearly, drift and naive do not do well with this time series. We will remove and review the other models.

```
pipes_fc |>
  filter(.model %in% c('arima', 'ets', 'mean')) |>
  autoplot(pipe_test)
```



These look more reasonable, so let's review the accuracy to make a more quantitative decision.

```
pipes_fc |>
  accuracy(pipes_transformed) |>
  select(-all_of(exclude_metrics)) |>
  kable()
```

.model	.type	RMSE	MASE	ACF1
arima	Test	0.5806040	0.8872433	0.4779361
drift	Test	0.5955503	0.9322190	0.4759054
ets	Test	0.5856248	0.8974954	0.4818509
mean	Test	0.5825069	0.8892324	0.4818509
naive	Test	0.6081722	0.9581620	0.4818509

Our models all improve upon a naive selection of the previous value given the $MASE < 1$. The best performers were the ARIMA model, and surprisingly, the simple mean. Given the random noise nature of the series, it is unsurprising that the (0,1,3) ARIMA model, heavily weight on the local moving average, and simple mean of the test series performed the best. Given the slight tradeoff in accuracy, we should prefer the very simple mean to forecast these series.

```

# Export forecasts with point estimates and prediction intervals
pipes_fc |>
  filter(.model %in% c('arima', 'mean')) |>
  hilo(level = c(80, 95)) |>
  unpack_hilo(c(`80%`, `95%`)) |> # unpacks intervals into separate columns
  as_tibble() |>
  select(datetime_hour, .model, .mean, `80%_lower`, `80%_upper`, `95%_lower`, `95%_upper`) |>
  write_xlsx("water_flow_forecast.xlsx")

```